

Lab Assignment 3

Due May 10, 2026 at 23:59:59

1 Objective

The purpose of this lab is to implement a **streamed tiled convolution** routine that uses both CUDA **streams** (with pinned host memory) and **shared memory** for on-chip tiling.

2 Instructions

The code template in `template_convolution.cu` provides the skeleton with basic import/export, variable declaration and stream setup. Students are expected to insert their code demarcated with `//@@ INSERT CODE HERE`. Specifically, you should:

- Allocate device memory
- Allocate pinned buffers for the input and output image
- Create and configure multiple `cudaStream_t` streams
- Partition the image into row-segments per stream
- Asynchronously copy each segment of the pinned input image to device
- Initialize thread block and grid dimensions
- Invoke the CUDA kernel on **each stream with the per-stream rowOffset**
- Asynchronously copy each computed segment back to the pinned output buffer
- Synchronize streams and copy from pinned output to host image data
- Free all device, pinned, and stream resources
- Write the CUDA kernel using `__shared__` memory to load a tile per channel

A basic convolution algorithm is provided in the kernel; you may modify and extend it as needed. The shared-memory tile is a $(\text{TILE_WIDTH} + \text{Mask_width} - 1) \times (\text{TILE_WIDTH} + \text{Mask_width} - 1)$ buffer per channel, sized so that **all halo pixels needed by one block are loaded once**.

Compile the template with the provided `Makefile`. The executable generated as a result of compilation can be run using the following command:

```
./Convolution_template -e <expected.ppm> \  
  -i <input0.ppm>,<input1.raw> \  
  -o <output.ppm> -t image
```

where `<expected.ppm>` is the expected output, `<input0.ppm>`, `<input1.raw>` are the input image and 5 x 5 mask, and `<output.ppm>` is an optional path to store the results.

You can also simply compile and run the code using the helper script:

```
bash test.sh -dataset <number>
```

All GPU compilation and execution must be performed on a compute node via SLURM. Activate the conda

environment first, then prefix every `nvcc/run` command with `srun -p class2 -N 1 --gres=gpu:1`. The login node has no GPU.

```
conda activate comp2
srun -p class2 -N 1 --gres=gpu:1 <command>
```

`README.md` has details on how to build `libgputk` and `template_convolution.cu`, as well as how to generate the input datasets with the provided `dataset_generator_convolution`.

3 What to Turn in

Submit a `report.pdf` with your `template_convolution.cu` and program execution logs (`.log` files) as a compressed `20xx-xxxxx.tar.gz` file. Your report should include the following:

1. How many floating point operations are being performed by your kernel per output pixel?
2. A brief explanation of how you partitioned the image and configured streams.
3. An analysis of how pinned host memory and streams overlap data transfer and compute.
4. Describe what further optimizations can be implemented to your kernel to achieve a performance speedup.
5. Explanation about your version of `template_convolution.cu` including the convolution kernel.
6. Execution times of the kernel with the provided input data. Please include the system information where you performed your evaluation and output the program execution log to `.log` files. For time measurement, use `gpuTKTime_start` and `gpuTKTime_stop` functions (details in `libgputk/README.md`).
7. Measured execution times (use `gpuTKTime_start/stop`) at dataset 6 for: (a) varying number of streams: 1, 2, 4, 8; (b) varying tile widths: 8, 16, 24, 32. Present results in a table or plot, and include system specifications and program execution logs. You can use `bash test.sh -dataset 6` for simpler testing.